GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 07.1.
PÁGINAS WEB MULTI-IDIOMA CON JSON Y LOCALSTORAGE

LM
Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es
2025/2026
Versión:260406.1319

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Objetivo..... 5
2. Introducción..... 5
3. Conceptos Clave..... 5
3.1 Archivos JSON para Traducciones..... 5
3.2 LocalStorage..... 6
4. Implementación Paso a Paso..... 6
4.1 Estructura HTML..... 6
4.2 Lógica JavaScript..... 7
4.3 Explicación del código:..... 9

UD07.1. PÁGINAS WEB MULTI-IDIOMA CON JSON Y LOCALSTORAGE

1. OBJETIVO

Crear una página web multi-idioma es crucial para llegar a una audiencia global. En estos apuntes, exploraremos cómo implementar esta funcionalidad utilizando **JSON** para almacenar las traducciones y **LocalStorage** para guardar la preferencia de idioma del usuario.

2. INTRODUCCIÓN

Tradicionalmente, las páginas multi-idioma se han construido con lógica compleja en el backend o con múltiples archivos HTML para cada idioma. Sin embargo, con las capacidades actuales de JavaScript, podemos manejar las traducciones directamente en el frontend, ofreciendo una experiencia más fluida y eficiente.

Utilizar **JSON** para las traducciones nos permite organizar de forma estructurada el contenido en diferentes idiomas, mientras que **LocalStorage** nos ayuda a recordar el idioma seleccionado por el usuario entre visitas, mejorando la usabilidad.

3. CONCEPTOS CLAVE

Para construir nuestra página multi-idioma, necesitamos entender algunos conceptos fundamentales:

3.1 Archivos JSON para Traducciones

Un archivo **JSON** (JavaScript Object Notation) es un formato ligero para el intercambio de datos. Lo utilizaremos para almacenar todas las cadenas de texto de nuestra web, organizadas por idioma y por claves.

Importante: La estructura del JSON debe ser consistente para facilitar el acceso a las traducciones.

```
{
"es": {
  "welcome_message": "¡Bienvenido a nuestra web!",
  "about_us": "Sobre nosotros",
  "contact": "Contacto"
},
"en": {
  "welcome_message": "Welcome to our website!",
```

```
    "about_us": "About Us",
    "contact": "Contact"
  },
  "fr": {
    "welcome_message": "Bienvenue sur notre site web !",
    "about_us": "À propos de nous",
    "contact": "Contact"
  }
}
```

En este ejemplo, cada idioma (`es`, `en`, `fr`) es una clave de nivel superior, y dentro de cada una, tenemos pares clave-valor (`welcome_message`, `about_us`, `contact`) que corresponden a las cadenas de texto traducidas.

3.2 LocalStorage

LocalStorage es un mecanismo de almacenamiento web que permite a las aplicaciones web almacenar datos en el navegador del usuario de forma persistente. A diferencia de las cookies, tiene una mayor capacidad de almacenamiento (típicamente 5MB o más) y los datos no se envían al servidor con cada solicitud.

Ojo: Los datos en LocalStorage no tienen fecha de caducidad y se mantienen hasta que el usuario los borra manualmente o se borra la caché del navegador.

Métodos principales de LocalStorage:

- `localStorage.setItem(key, value)`: Almacena un par clave-valor.
- `localStorage.getItem(key)`: Recupera el valor asociado a una clave.
- `localStorage.removeItem(key)`: Elimina un par clave-valor.
- `localStorage.clear()`: Elimina todos los pares clave-valor.

4. IMPLEMENTACIÓN PASO A PASO

Ahora, veamos cómo integrar estos conceptos en nuestra página web.

4.1 Estructura HTML

Nuestro HTML necesitará elementos donde insertaremos las traducciones, y un selector para cambiar el idioma.

```
<!DOCTYPE html>
<html lang="es">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Página Multi-idioma</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <select id="language-selector">
      <option value="es">Español</option>
      <option value="en">English</option>
      <option value="fr">Français</option>
    </select>
    <h1 data-key="welcome_message"></h1>
  </header>
  <nav>
    <ul>
      <li><a href="#" data-key="about_us"></a></li>
      <li><a href="#" data-key="contact"></a></li>
    </ul>
  </nav>
  <main>
    <p data-key="paragraph_one"></p>
    <p data-key="paragraph_two"></p>
  </main>
  <footer>
    <p data-key="footer_text"></p>
  </footer>

  <script src="translations.json"></script>
  <script src="script.js"></script>
</body>
</html>
```

Observa el atributo **`data-key`** en los elementos HTML. Este atributo nos permitirá identificar qué texto debe ser traducido y con qué clave buscarlo en nuestro archivo JSON.

4.2 Lógica JavaScript

Aquí es donde conectamos el JSON con el HTML y LocalStorage.

```
// script.js

// usamos fetch() para hacer peticiones de red, en este caso lo hace sobre nuestro propio servidor.
let translations = {}; // Almacenará los datos de translations.json
const languageSelector = document.getElementById('language-selector');
```

```
// Función para cargar las traducciones
async function loadTranslations() {
    try {
        const response = await fetch('translations.json');
        translations = await response.json();
        initializeLanguage();
    } catch (error) {
        console.error('Error al cargar las traducciones:', error);
    }
}

// Función para aplicar las traducciones a la página
function applyTranslations(lang) {
    document.querySelectorAll('[data-key]').forEach(element => {
        const key = element.dataset.key;
        if (translations[lang] && translations[lang][key]) {
            element.textContent = translations[lang][key];
        } else {
            console.warn(`Traducción no encontrada para la clave: ${key} en idioma: ${lang}`);
        }
    });
    // Actualizar el atributo lang del HTML para accesibilidad
    document.documentElement.lang = lang;
}

// Función para inicializar el idioma al cargar la página
function initializeLanguage() {
    const savedLang = localStorage.getItem('selectedLang');
    const defaultLang = 'es'; // Idioma por defecto si no hay nada guardado
    let currentLang = savedLang || defaultLang;

    // Asegurarse de que el idioma guardado exista en nuestras traducciones
    if (!translations[currentLang]) {
        currentLang = defaultLang;
    }

    languageSelector.value = currentLang;
    applyTranslations(currentLang);
}

// Event listener para el cambio de idioma
languageSelector.addEventListener('change', (event) => {
    const newLang = event.target.value;
    localStorage.setItem('selectedLang', newLang); // Guardar la preferencia en
LocalStorage
    applyTranslations(newLang);
});

// Cargar las traducciones cuando el DOM esté listo
```

```
document.addEventListener('DOMContentLoaded', loadTranslations);
```

4.3 Explicación del código:

1. **loadTranslations()**:
   - Utiliza 'translations.json' para cargar el archivo JSON de forma asíncrona.
   - Una vez cargado, guarda los datos en la variable translations e inmediatamente llama a initializeLanguage().
   - Incluye un try-catch para manejar posibles errores de carga.
2. **applyTranslations(lang)**:
   - Selecciona todos los elementos con el atributo data-key en el HTML.
   - Para cada elemento, obtiene el valor de data-key (que es nuestra clave de traducción).
   - Accede al objeto translations utilizando el idioma (lang) y la clave (key) para obtener el texto traducido.
   - Asigna el texto traducido al textContent del elemento.
   - Actualiza el atributo lang del elemento <html> para una mejor accesibilidad y SEO.
   - **Importante:** Se incluye una comprobación if (translations[lang] && translations[lang][key]) y un console.warn para alertar si una traducción no se encuentra, lo que ayudará en la depuración.
3. **initializeLanguage()**:
   - Intenta obtener el idioma guardado en localStorage usando localStorage.getItem('selectedLang').
   - Si no hay idioma guardado, usa 'es' como idioma por defecto.
   - Asegura que el idioma seleccionado (guardado o por defecto) realmente existe en nuestro objeto translations. Si no, se vuelve a 'es'.
   - Establece el valor del <select> al idioma actual.
   - Llama a applyTranslations() con el idioma determinado para mostrar el contenido correcto.
4. **languageSelector.addEventListener('change', ...)**:
   - Cuando el usuario cambia la opción en el selector, este evento se dispara.
   - Se obtiene el nuevo idioma seleccionado (event.target.value).
   - Se guarda la preferencia del usuario en localStorage con localStorage.setItem('selectedLang', newLang).
   - Se llama a applyTranslations() para actualizar el contenido de la página.
5. **document.addEventListener('DOMContentLoaded', loadTranslations)**:
   - Asegura que las traducciones se carguen solo cuando todo el HTML ha sido analizado y está listo para ser manipulado por JavaScript.